

CUDA Cross-Entropy and Chunk-Size Trade-offs for Transformer Training on a T4

Irtiqa Haider

23 September 2026 | Version 1.0.0

<https://github.com/IrtiqaHaider/budgetce>

Abstract. BudgetCE evaluates a memory-bounded linear cross-entropy implementation on a single NVIDIA Tesla T4. A custom CUDA row-reduction kernel is integrated into a recomputing PyTorch autograd path, while matrix products use the existing backend. The initial experiment measures six held-out operator shapes, three candidate CUDA chunk sizes, native and chunked PyTorch baselines, and a calibrated execution selector. At the largest operator shape, the fixed 1,024-row CUDA chunk reduces extra peak allocated memory from 3,140 to 392 MiB and provides $1.20\times$ the native forward/backward throughput. A subsequent, separately analyzed confirmation compares both chunked paths at the same chunk size in a 20.19M-parameter decoder. Across three 100-update trials per method at sequence length 1,024, CUDA provides 14.8% higher training throughput and 32.3% lower peak allocated memory than native, and 31.6% higher throughput than the project’s matched-chunk PyTorch baseline. All recorded numerical gates pass; no intended optimizer updates are skipped. The learned selector makes the same choices as the simpler largest-predicted-feasible-chunk rule, so no adaptive-policy advantage is claimed. Results characterize the recorded hardware, software, and synthetic workloads rather than convergence quality, distributed scaling, or state-of-the-art library performance.

1 Purpose and contribution

The output projection of a language model can create a large token-by-vocabulary logits matrix. Retaining full-size intermediates through backward can consume memory unrelated to the decoder’s useful hidden representations. Memory-efficient cross-entropy is established prior work, notably Cut Cross-Entropy and Liger Kernel [1, 2]. This report does not claim to invent the objective or outperform those libraries.

The contribution is an inspectable T4-compatible implementation, a controlled comparison of chunking and backend effects, and a reproducible evaluation of a budget-constrained selector. The questions are whether custom CUDA adds value beyond chunking, whether operator improvements survive whole-model integration, and whether learned cost prediction changes execution choices.

Matched training method	Tokens/s	Step time (ms)	Peak (MiB)
Native PyTorch	40,593	50.45	841.33
PyTorch, chunk 1,024	35,411	57.84	809.84
CUDA, chunk 1,024	46,607	43.94	569.84

Table 1: Targeted confirmation: three repeats per method; 20 warmup and 100 measured optimizer updates per trial. Memory is PyTorch allocated memory.

Source: Recomputed from the nine raw confirmation records; `results/matched_training.csv`.

2 Operator design

Let $H \in \mathbb{R}^{N \times D}$ be hidden representations and $W \in \mathbb{R}^{V \times D}$ classifier weights. The native path materializes $Z = HW^\top$. For valid targets y_i , mean cross-entropy is

$$L = \frac{1}{N_{\text{valid}}} \sum_{i \in \mathcal{I}} \left[\log \sum_{j=1}^V \exp(Z_{ij}) - Z_{i, y_i} \right]. \quad (1)$$

A $4,096 \times 65,536$ logits tensor alone occupies 512 MiB in FP16 or 1,024 MiB in FP32. These are calculated storage sizes, not measured training peaks. Splitting token rows into chunks of size C reduces the logits workspace to $\min(N, C) \times V$; it does not eliminate weight, gradient, or cast storage.

2.1 Forward and backward

The custom autograd function computes each projection chunk, reduces row losses, and releases the chunk. It retains the cast hidden states, classifier weights, labels, and normalization scalar, not every chunk’s autograd graph. Backward recomputes projection chunks, transforms disposable logits into scaled gradient values in place, and computes

$$\frac{\partial L}{\partial H} = GW, \quad \frac{\partial L}{\partial W} = G^\top H, \quad G_{ij} = \frac{\text{softmax}(Z_i)_j - \mathbf{1}[j = y_i]}{N_{\text{valid}}}. \quad (2)$$

The upstream gradient scale is applied before the matrix products. Classifier-gradient partials are accumulated into FP32 storage. Matrix multiplication is still implemented by the PyTorch/backend path; the CUDA contribution is the row-wise reduction and gradient transformation, not a new GEMM kernel.

2.2 CUDA implementation

One CUDA block processes one row using 256 threads. Warp shuffles and shared-memory reductions compute the row maximum and exponential sum in FP32. The row loss uses a stabilized log-sum-exp form. Backward writes scaled softmax-minus-target values into the disposable logits tensor, avoiding an additional full FP32 probability tensor for that chunk. Launches use PyTorch’s current CUDA stream and an explicit device guard. The JIT extension is compiled for the actual device architecture without a fast-math flag.

This source-level allocation difference is an implementation explanation, not a hardware-counter finding about bandwidth or occupancy. The experiment did not collect profiler traces or memory-controller counters.

2.3 Contract and numerical limits

The public wrapper accepts contiguous hidden/weight tensors and int64 labels on one device, supports mean or sum reduction and an ignore index, and checks target bounds. An all-ignored mean deliberately returns zero. Label smoothing, class weights, higher-order gradients, and noncontiguous layout handling are outside the supported contract. CUDA compute supports FP16 and FP32; the measured study fixes FP16 compute with FP32 parameters and accumulation.

No vocabulary classes or intentionally small gradient contributions are filtered. Nevertheless, FP16 GEMM results and chunk-wise accumulation are rounded: the numerical claim is first-order agreement within declared tolerances, not bitwise equivalence. Different floating-point evaluation orders can give different results [6].

2.4 Baselines

The native baseline uses the same output projection followed by FP32 cross-entropy. The PyTorch chunked baseline shares the recomputing autograd structure but uses PyTorch probability operations. The initial experiment compares both chunked backends at $C = 256$; the targeted training confirmation also matches both at $C = 1,024$. This prevents attributing every difference caused by chunk size to custom CUDA.

3 Experimental method

Control	Recorded setting
Device	Tesla T4; compute capability 7.5; 14.56 GiB reported memory
Software	PyTorch 2.11.0+cu128; CUDA toolkit 12.8.93; driver 580.82.07; Python 3.13.15
Operator precision	FP16 matrix compute; FP32 row reductions and classifier-gradient accumulation; static scale 128
GPU/CPU settings	TF32 disabled; FP16 reduced-precision reduction disabled; two CPU threads
Calibration shapes	$N \in \{256, 1, 024\}$, $V \in \{32, 768, 65, 536\}$, $D = 512$
Held-out shapes	$N \in \{512, 2, 048, 4, 096\}$; same two vocabularies; $D = 512$
Plans	Native; PyTorch $C = 256$; CUDA $C \in \{64, 256, 1, 024\}$
Operator allowances	384 and 768 MiB of <i>extra</i> allocated memory above resident inputs
Decoder	20,189,696 parameters; four blocks; width 256; four heads; vocabulary 32,768; untied classifier
Training	AdamW, learning rate 0.001, FP16 autocast; microbatch one, accumulation two; fixed MATH attention

Table 2: Recorded controls. No device-clock locking or process-isolation telemetry was retained.

3.1 Two campaigns, analyzed separately

The initial study contains 20 calibration blocks, 90 held-out operator blocks, and 18 short whole-training trials. Each operator block has one warmup plus three measured forward/backward calls. Training uses sequences 512 and 1,024 with two warmup and ten measured optimizer updates per trial. Calibration and held-out token counts are disjoint; decisions are frozen before evaluation.

After inspecting the initial results, a targeted confirmation fixes sequence length 1,024 and compares native, PyTorch:1024 and CUDA:1024. Three repeats per method yield nine trials, 900 measured optimizer updates, and 1,843,200 measured tokens. Each trial has 20 additional warmup updates. Methods are shuffled within each repeat; model and input seeds are paired across methods. The saved environment matches the initial campaign. This follow-up was selected post hoc, not independently preregistered. Its longer windows are not pooled with the original windows.

3.2 Timing and memory

Inputs are prepared before timing. CUDA synchronization bounds each wall-clock block; CUDA-event intervals are recorded separately. Validation, compilation, garbage collection, and warmup are outside the measured window. Whole-training throughput is

$$\text{tokens/s} = \frac{\sum_r \text{measured tokens}_r}{\sum_r \text{measured wall seconds}_r}. \quad (3)$$

It is not an arithmetic average of per-trial rates. The recorded block ranges are 15.6–632.3 ms for initial held-out operators, 0.254–0.659 s for initial training, and 4.374–5.811 s for matched training. The longer confirmation improves the measurement window but still is not a long-duration service study. Warmup and repeated observations are important benchmark practices [4].

Allocated peaks are reset after warmup. Extra operator memory subtracts resident-input allocation; whole-training peaks do not. Reserved memory and before/after NVML snapshots are distinct fields. PyTorch allocated memory measures tensor allocations, not all device usage [3]. AMP updates are counted using optimizer hooks so skipped updates cannot masquerade as completed work [5].

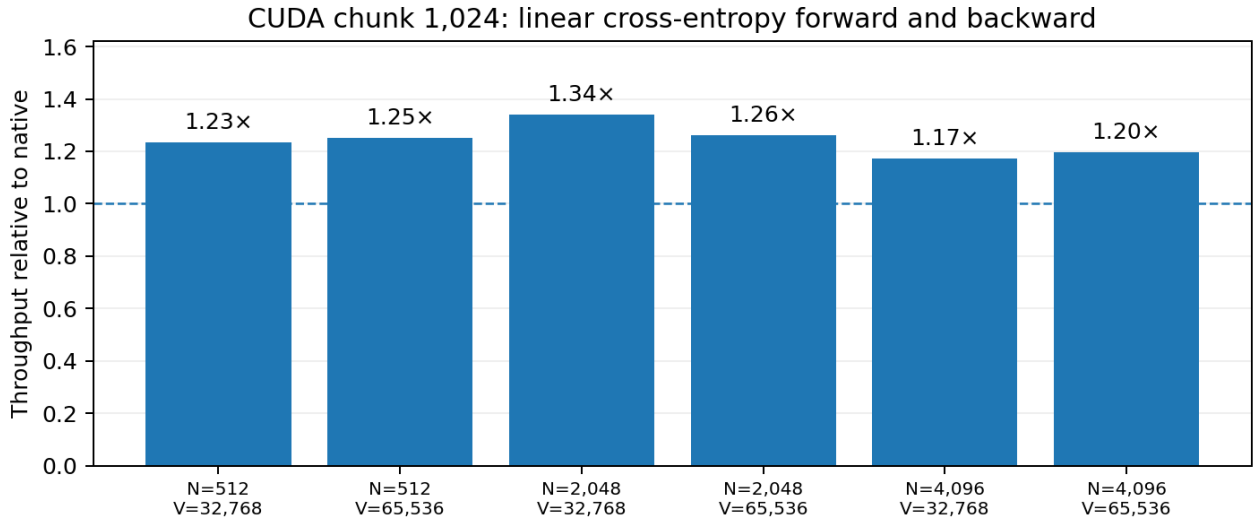
4 Operator results

N	V	Native ms	CUDA ms $C = 1,024$	Ratio	Native extra MiB	CUDA extra MiB
512	32,768	6.60	5.34	$1.235\times$	224.50	161.00
512	65,536	13.35	10.66	$1.252\times$	448.50	321.00
2,048	32,768	19.25	14.36	$1.340\times$	802.00	196.00
2,048	65,536	40.23	31.89	$1.261\times$	1,602.00	388.00
4,096	32,768	38.11	32.52	$1.172\times$	1,572.00	200.00
4,096	65,536	80.01	66.80	$1.198\times$	3,140.00	392.00

Table 3: Complete projection, loss and backward; no optimizer. $D = 512$. Ratios are native time divided by CUDA time. One fixed CUDA candidate is shown across all shapes.

Source: `results/operator_comparisons.csv`; three measured blocks per plan and shape.

The fixed CUDA:1024 path provides 1.17–1.34 $\times$ native operator throughput across the six held-out shapes. At the largest shape, extra peak allocation falls by 87.5%. Total allocated peak for that case falls from approximately 3,292.28 to 544.28 MiB, or 83.5%. Neither percentage is a whole-transformer memory reduction.



Hidden width 512; pooled three-block timings. Chunk 1,024 is not feasible under every memory allowance.

Figure 1: The fixed $C = 1,024$ path improves measured operator throughput, but is not feasible under every requested memory allowance.

4.1 Matched chunks isolate an implementation contribution

At $C = 256$, CUDA is 1.55–1.69 $\times$ faster than the project’s PyTorch chunked path across the same six shapes. For $N = 4,096, V = 65,536$, PyTorch:256 requires 141.65 ms and 360.01 MiB of extra allocation, whereas CUDA:256 requires 91.23 ms and 296.00 MiB. These are complete operator-path comparisons, not isolated CUDA-kernel timings or comparisons with all available CE libraries.

4.2 The tighter budget changes the answer

CUDA:1024 uses 388 and 392 MiB on the two largest-token, large-vocabulary shapes, exceeding the 384 MiB allowance. The selector therefore chooses CUDA:256. These selected plans satisfy the measured allowance but have approximately 11–12% lower throughput than native; native itself exceeds that allowance. The result is a feasible memory/performance trade-off rather than an unconditional improvement. All configurations physically fit the T4: the experiment did not demonstrate a formerly impossible model size or actual-device out-of-memory avoidance.

5 Longer matched-chunk training confirmation

All nine trials completed successfully. Every intended optimizer update ran: 900 measured plus 180 warmup, with finite recorded losses and final parameters. Both shape-matched preflights passed. The follow-up changes the measurement window and PyTorch comparison chunk, not the CUDA kernel, optimizer, model architecture, attention backend, or gradient tolerances.

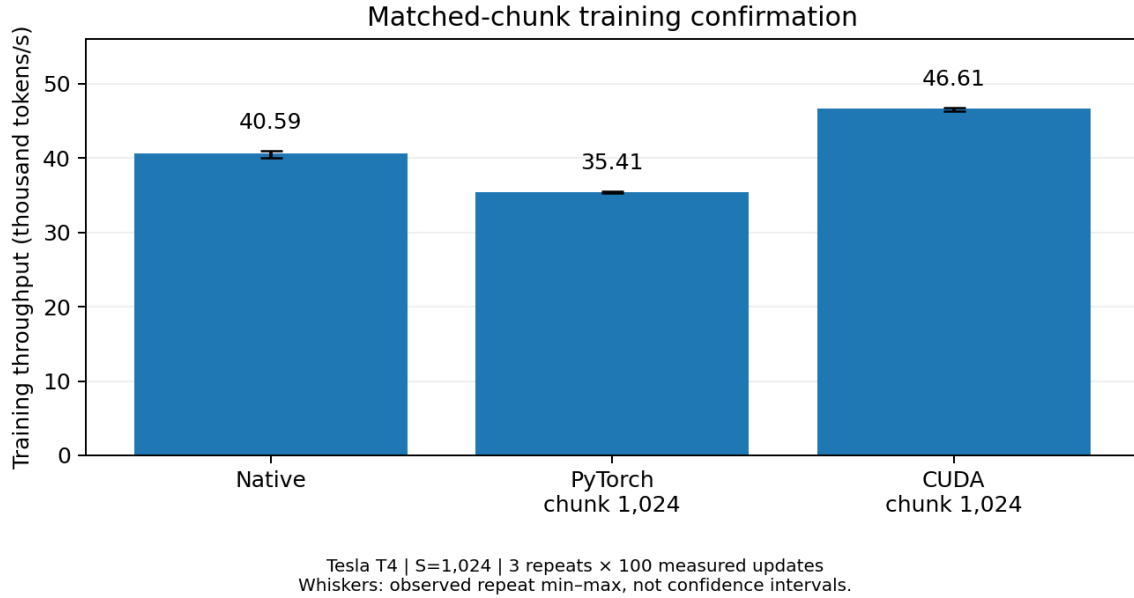


Figure 2: Pooled training throughput. Error bars show the minimum and maximum of three observed trial rates, not confidence intervals.

CUDA:1024 reaches 46,607 tokens/s versus 40,593 for native and 35,411 for PyTorch:1024. Relative to native this is **14.8% higher throughput**, equivalently **12.9% less time per optimizer update**. Those percentages describe different denominators and must not be interchanged. Relative to the matched PyTorch chunk, throughput is 31.6% higher.

Method	Step ms	Repeat min-max ms	Sample CV	Peak MiB
Native	50.452	50.014–51.095	1.13%	841.334
PyTorch:1024	57.835	57.545–58.107	0.49%	809.836
CUDA:1024	43.941	43.738–44.218	0.56%	569.836

Table 4: Three trial means per method. Pooled step time weights by measured update count; sample CV is descriptive variation of the three trial means.

Peak whole-training allocation falls 32.3% versus native and 29.6% versus the matched PyTorch chunk. At $S = C = 1,024$, each microbatch fits into a single row chunk. Thus, the CUDA-versus-PyTorch difference here cannot be explained by giving CUDA more chunks or a different microbatch. The source’s lower-intermediate-allocation path is consistent with the measured difference, but detailed kernel-level causal attribution would require profiling.

Repeat-paired CUDA/native throughput ratios are 1.1435, 1.1454, and 1.1555. Ratios versus matched PyTorch are 1.3157, 1.3188, and 1.3141. All are above one in this session; no formal significance or cross-session reliability claim follows from three repeats.

The largest absolute difference between CUDA and native recorded loss trajectories is 0.02199 across the 120-update histories. The actual-shape preflight maximum relative gradient error is 7.83×10^{-6} , below its 5×10^{-3} tolerance. These are different measurements: a passing local gradient check does not imply identical multi-step trajectories or preserved convergence quality.

6 Execution selection and numerical validation

6.1 Learned selection offers no measured advantage

For each CUDA candidate, the planner fits a three-feature nonnegative model: constant cost, $NVD/10^9$, and $[N/\min(N, C)]$. Its workspace estimate includes casts, gradient outputs and accumulators, a weight-gradient temporary, and chunk storage, with a calibrated multiplier and safety margin. This is a prediction, not an allocator-enforced limit.

All twelve held-out shape/budget decisions satisfy their measured allowance and choose the best observed feasible candidate among the three CUDA choices. However, every decision is identical to the largest-predicted-feasible-chunk rule. The learned policy therefore has **no measured selection or speed advantage** over that simpler rule. These twelve decisions reuse six candidate timing sets; they are not twelve independent online adaptive executions.

The recorded calibration cost is 2,582.31 ms. A break-even calculation against an arbitrarily small fixed chunk cannot justify learning when a stronger simple rule selects the same plan. Additionally, every $C = 1,024$ calibration shape uses one chunk, making the constant and chunk-count columns identical; that model has rank two rather than three. Its coefficients should not be interpreted as separately identified physical costs. End-to-end width 256 is also outside the width-512 calibration set. The final confirmation fixes the CUDA candidate and claims no policy credit.

6.2 Validation coverage

Evidence	Outcome
Initial raw blocks: calibration / evaluation / training	20 / 90 / 18 successful
Initial CUDA numerical gate	21 / 21 checks passed
Additional matched-shape numerical gate	2 / 2 checks passed
Confirmation trials	9 / 9 successful
Confirmation optimizer updates: measured / warmup	900 / 180, no skips
Independent offline arithmetic and integrity audit	64 / 64 checks passed
Release CPU tests: original / additional	38 / 10 passed

Table 5: Recorded GPU evidence and locally executed CPU checks are distinct. Packaging did not independently rerun the GPU experiment.

The initial gate checks relative losses and gradients, ignored labels, irregular shapes, accumulation, the full model, and nondefault streams. Its maximum recorded relative gradient error is approximately 2.93×10^{-4} , below the applicable declared tolerances. First-order numerical checks, finite final parameters, and counted optimizer steps are necessary controls, not evidence of real-language quality or convergence.

7 Related work and positioning

Cut Cross-Entropy addresses large-vocabulary loss memory without materializing the entire logits matrix, and discusses filtering negligible gradient contributions [1]. Liger Kernel studies fused and chunked Triton kernels for LLM training [2]. BudgetCE does not compare against either implementation on identical hardware. It implements a narrower CUDA/PyTorch path, retains dense vocabulary contributions, and measures it on a constrained T4. The contribution is engineering, reproducible evaluation, and an explicit negative result for learned selection, not a new cross-entropy objective, an exhaustive optimizer, or a state-of-the-art claim.

8 Limitations and reproducibility

Hardware and workloads. One recorded GPU/software environment, synthetic tokens and hidden representations, one small decoder, and limited shapes were evaluated. Attention is deliberately fixed to PyTorch’s MATH backend. These results do not establish behavior with other attention kernels, architectures, GPUs, distributed collectives, long contexts, or production traffic. All methods fit physically on the T4.

Duration and uncertainty. The targeted windows are longer than the initial windows but remain approximately four to six seconds each, in one session. There are only three trial means per condition. Min–max ranges and sample CV are not confidence intervals; no significance tests are reported. The short $S = 512$ campaign has a repeat that reverses CUDA’s throughput ordering and is not a robust speedup claim. Both campaigns are retained and kept separate.

Numerics and mechanisms. No convergence, perplexity, downstream-task quality, or bitwise-equivalence claim is made. GPU clocks, power, full device-memory time series, and foreign-process isolation were not recorded. No profiler traces were collected. Kernel bandwidth, thermal throttling, occupancy, and bottleneck mechanisms therefore remain unmeasured. Allocated memory omits non-PyTorch allocations and is not an exact whole-device footprint.

Evidence packaging. The initial compact archive preserves all 128 raw blocks and relevant metadata/source bytes, while removing redundant plots, preview reports and documentation. It carries a new subset manifest plus the original parent hash. The matched archive is unchanged. The compiled report is distributed without a LaTeX bundle. The runnable code and data checks remain available independently of the report.

8.1 Reproducing the analysis

The repository supplies CPU-only hash/arithmetic checks and data-derived figures. From its root:

```
python scripts/verify_release.py
python scripts/reproduce.py -out reanalysis
python scripts/plot_results.py -results reanalysis
```

The repository-backed notebook defaults to this recorded-data audit. New GPU execution is explicit and requires CUDA-enabled PyTorch plus a compatible compiler. It runs the numerical gate before calibration or measurement, then the same targeted runner for the longer comparison. New results receive separate output directories and environment identities. The full commands are in `REPRODUCE.md`.

Provenance. Engine SHA-256 prefix: 858c1389d6b33490. Matched-runner prefix: 645ea36aa3399550. Full identifiers, original archive hashes, and packaged-archive hashes are stored in `data/provenance.json` and the result manifests. Offline source and arithmetic checks do not independently verify physical execution or unavailable tensor contents.

9 Conclusion

BudgetCE demonstrates a useful, bounded implementation trade-off. Custom CUDA row reductions with chunked recomputation improve complete linear-cross-entropy forward/backward performance over the native and project-specific chunked baselines. In the longer matched-chunk transformer confirmation, the CUDA path provides **14.8% higher throughput and 32.3% lower peak allocated memory than native**. The simpler feasible-chunk policy matches the learned selector. The report therefore attributes the measured benefit to the evaluated implementation and execution plan, not to an unproven adaptive algorithm or universal training optimization.

A Initial short-training results

Sequence	Method	Tokens/s	Step ms	Peak MiB
512	Native	36,740	27.87	576.71
512	PyTorch:256	30,914	33.12	496.97
512	CUDA:1024	37,240	27.50	464.96
1,024	Native	39,715	51.57	839.47
1,024	PyTorch:256	32,178	63.65	567.97
1,024	CUDA:1024	46,066	44.46	567.97

Table 6: Original campaign: two warmup plus ten measured optimizer updates per trial, three repeats per condition. These values are not pooled with the targeted confirmation.

The initial $S = 1,024$ CUDA/native throughput ratio was 1.160; the later, longer matched-window estimate is 1.148. The first campaign compared PyTorch:256 with CUDA:1024 and used shorter input banks. The later campaign controls the chunk-size comparison and changes timing-window length, so cross-campaign memory or timing differences must not be attributed solely to the backend.

References

- [1] E. Wijmans, B. Huval, A. Hertzberg, V. Koltun, and P. Krähenbühl. *Cut Your Losses in Large-Vocabulary Language Models*. arXiv:2411.09009, 2024, revised 2025. <https://arxiv.org/abs/2411.09009>.
- [2] P.-L. Hsu et al. *Liger Kernel: Efficient Triton Kernels for LLM Training*. arXiv:2410.10989, 2024, revised 2025. <https://arxiv.org/abs/2410.10989>.
- [3] PyTorch. *torch.cuda.memory.max_memory_allocated*. Version 2.11 documentation. https://docs.pytorch.org/docs/2.11/generated/torch.cuda.memory.max_memory_allocated.html. Accessed 23 September 2026.
- [4] PyTorch. *Benchmark Utils*. Version 2.11 documentation. https://docs.pytorch.org/docs/2.11/benchmark_utils.html. Accessed 23 September 2026.
- [5] PyTorch. *Automatic Mixed Precision Examples*. Version 2.11 documentation. https://docs.pytorch.org/docs/2.11/notes/amp_examples.html. Accessed 23 September 2026.
- [6] PyTorch. *Numerical Accuracy*. Version 2.11 documentation. https://docs.pytorch.org/docs/2.11/notes/numerical_accuracy.html. Accessed 23 September 2026.

Artifact availability. Code, measurements, tests and this compiled report: <https://github.com/IrtiqaHaider/budgetce>. All experimental numbers are derived from the included measurement records, not copied from external publications. This is an independent technical project report; no publication venue or institutional affiliation is asserted.